IT-305 : COMPUTER NETWORKS : SOCKET PROGRAMMING PROJECT

Design Document

Fault-Tolerant, Topic-Based File Distribution Service over TCP Sockets, with Checkpointed Resume and Parallel Transfer

Dhirubhai Ambani Institute of Information and Communication Technology (DA-IICT)

IT-305 Computer Networks Laboratory, Autumn Semester 2026

Project Group 040_093 (merged from Lab Group 040 and Lab Group 093)

Design Document Submission: 27 September 2026

Table of Contents

1. Introduction

2. Project Objectives

3. Dataset and Topic Organization

4. System Architecture Overview

5. Part I: Basic Topic-Based File Distribution

6. Part II: Fault-Tolerant File Transfer

7. Consolidated Performance Measurement Plan

8. Program Structure and Additional Features

9. Design Thought Process and Alternatives Considered

10. Assumptions, Limitations and Edge Cases

11. Build, Execution and CLI Compliance

12. Conclusion

1. Introduction

This document specifies the design of a TCP socket-based, topic-oriented file distribution service built for the IT-305 socket programming project. A server organises files into topics, one topic per sub-directory of a supplied root directory, and streams every file belonging to a requested topic to a connecting client.

The project has two parts. Part I implements the basic file-distribution protocol under a single-threaded and a multi-threaded server and compares their throughput as client concurrency increases. Part II extends the multi-threaded server with deliberate, probabilistic connection failures and studies three recovery strategies of increasing sophistication: no session state (Case 1), checkpoint-based resume (Case 2), and checkpoint-based resume over several parallel TCP connections (Case 2 Enhanced).

2. Project Objectives

- Build a reliable, TCP-based topic file-distribution service that satisfies the required GET <topic> request exactly as specified.
- Implement and fairly compare a single-threaded and a POSIX-pthread multi-threaded server (Part I).
- Measure aggregate transfer rate as a function of the number of concurrent clients.
- Design a fault-injection mechanism that lets the server probabilistically interrupt an active client connection mid-transfer.
- Design a session-management protocol (Case 2) so a client resumes an interrupted transfer from its last checkpoint instead of restarting from scratch.
- Design and evaluate a performance enhancement, parallel TCP connections per topic transfer, for Case 2 Enhanced.
- Quantify, across all Part II cases, the total bytes transmitted, useful bytes, redundant bytes, completion time and throughput.
- Keep every executable's command-line interface exactly as specified so the submission can be auto-graded.

3. Dataset and Topic Organization

The project uses the Animals-10 dataset. Each animal category is a sub-directory of the dataset root and is treated as one topic; the sub-directory name is used verbatim as the topic key. No topic name or directory path is hard-coded: the root directory is always supplied to the server on the command line, as required by the automated-testing interface.

```
topics_root_dir/
|-- cane/          -> topic "cane"
|     |-- img1.jpeg
```

```
|     `-- img2.jpeg
|-- cavallo/        -> topic "cavallo"
|-- elefante/  |-- farfalla/  |-- gallina/
|-- gatto/     |-- mucca/     |-- pecora/
`-- ragno/     `-- scoiattolo/
```

At start-up the server scans topics_root_dir once and records every immediate sub-directory as a topic. Adding a new topic to the dataset later requires no source-code change.

4. System Architecture Overview

The system follows a classical client-server model over TCP. The server listens on a configurable port, accepts connections, resolves the requested topic to a directory, and streams file metadata followed by binary file content using a fixed-length message framing protocol (see Section 5.2). The client connects, issues the request, and reconstructs the topic directory locally.

Component	Responsibility
Server	Accepts TCP connections; validates the requested topic; enumerates and streams its files; injects faults in Part II; maintains session/checkpoint state in Case 2 and Case 2 Enhanced.
Client	Connects (or reconnects); issues GET / RESUME / FILE_REQUEST as appropriate; receives file metadata and data; builds the output directory tree; detects a broken connection and drives recovery; records performance statistics.
Topic directory	Filesystem folder holding every file belonging to one topic; discovered dynamically, never hard-coded.
Worker thread	One POSIX thread per accepted connection, in the multi-threaded server and in every Part II case.
Session / checkpoint table	In-memory, mutex-protected server-side structure that lets a Case 2 / Enhanced client resume mid-file after reconnecting.

5. Part I: Basic Topic-Based File Distribution

Part I implements the base protocol required by the assignment and studies the effect of concurrency by comparing a single-threaded and a multi-threaded server.

5.1 Server Data Structure for Topic Mapping

At start-up the server scans topics_root_dir with a single readdir() pass and builds an in-memory array of topic records. An array, rather than a hash table, is used deliberately: the dataset has only ten topics, so a linear scan costs at most ten string comparisons per request, negligible next to file-transfer time; a hash table would add hashing and collision-handling complexity with no measurable benefit at this scale.

```
struct topic_entry {
    char name[MAX_TOPIC_LEN];   // e.g. "cane"
    char path[MAX_PATH_LEN];    // e.g. "<root>/cane"
};
struct topic_entry topic_table[MAX_TOPICS];
int topic_count;
```

Lookup: on GET <topic>, the server linearly scans topic_table for a matching name. On success it opens the associated directory and enumerates the regular files inside it (skipping ".", ".." and nested sub-directories) to build the file list for that transfer. On failure it returns ERROR UNKNOWN_TOPIC and closes the connection without starting a transfer.

Every fixed-length field parsed from an incoming message (topic name, filename, method code) is bounds-checked against its declared field width before use, and is always null-terminated within the destination buffer, so a malformed or over-length field can never overflow a fixed-size buffer on either side.

5.2 Client-Server Message Format: Fixed-Length Messages

The assignment mandates only the client's opening request, GET <topic>; every other message below is our own protocol design. We deliberately use fixed-length, fixed-layout binary messages rather than newline-terminated text lines. Every message on the wire is an instance of one fixed-size C struct, message_header, sent as exactly sizeof(message_header) bytes; the receiver always knows exactly how many bytes to read for the header before it even inspects the content, which removes any need to scan for a delimiter.

```
#define METHOD_LEN   8      // e.g. "GET", "OK", "FILE", "DONE", "ERROR"
#define TOPIC_LEN    64     // zero-padded topic name
#define FNAME_LEN    256    // zero-padded file name

struct message_header {
    char     method[METHOD_LEN];   // zero-padded ASCII method code
    char     topic[TOPIC_LEN];     // used by GET, OK, ERROR
    char     filename[FNAME_LEN];  // used by FILE
    uint32_t num_files;            // used by OK              (network byte order)
    uint64_t filesize;             // used by FILE            (network byte order)
    uint64_t offset;               // used by FILE / CHECKPOINT/ RESUME
};
```

Numeric fields are fixed-width integers converted with htonl()/htons() (and a portable 64-bit byte-swap helper for filesize and offset) before transmission, so field boundaries never depend on how the number is printed. A field that a given method does not use is simply left zero. Unused trailing bytes of the topic/filename fields are zero-padded so the struct size, and therefore the number of bytes to recv(), is identical for every message of a given method.

Message (method)	Direction	Purpose
GET	Client -> Server	Required. header.topic names the topic; requests every file belonging to it.
OK	Server -> Client	Topic exists; header.num_files announces how many files will follow.

Message (method)	Direction	Purpose
FILE	Server -> Client	Announces the next file: header.filename and header.filesize, followed immediately by exactly filesize raw bytes.
<raw file bytes>	Server -> Client	Exactly filesize bytes of that file's content, sent right after the FILE header (no separate framing needed since the length is already known).
DONE	Server -> Client	All files sent; server closes the connection.
ERROR	Server -> Client	header.topic carries UNKNOWN_TOPIC (as a fixed reason code); no transfer starts.

Normal transfer sequence for GET cane against a two-file topic: the client sends one GET header; the server replies OK (num_files = 2), then FILE + bytes for the first image, then FILE + bytes for the second image, then DONE.

5.3 Reliable Fixed-Length Framing

TCP delivers a continuous byte stream and does not preserve application-level message boundaries, so one send() is not guaranteed to arrive as one recv(). Every read or write of a known-length block, whether it is a fixed-size message_header or a fixed number of file bytes, therefore loops on send()/recv() until the required byte count has actually been transferred:

```
ssize_t send_all(int fd, const void *buf, size_t len) {
    size_t sent = 0;
    while (sent < len) {
        ssize_t n = send(fd, (char*)buf + sent, len - sent, 0);
        if (n <= 0) return -1;      // connection lost
        sent += (size_t) n;
    }
    return (ssize_t) sent;
}
// recv_all() is symmetric: loops until len bytes are read,
// treating recv() == 0 (peer closed) as a connection fault.
```

Because every message is a fixed-size struct, the receiver always calls recv_all(fd, &header, sizeof(header)) first and never has to search the stream for a delimiter; this is simpler and slightly cheaper than scanning for a newline, at the cost of a small amount of zero-padding overhead per message, which is negligible next to file-transfer time. Before sending a file, the server sends the fixed-size FILE header carrying the exact size, so the client always knows how many bytes to read for that file, independent of either side's internal buffer size.

5.4 Single-Threaded Server

The single-threaded server is the simplest correct implementation and forms the performance baseline required by the assignment.

- Create, bind and listen on the TCP socket given by <port>.
- accept() one client connection at a time.

- Read and validate the GET header on that connection.
- Transfer every file for the topic using the framing protocol above.
- Send DONE and close the connection, then only call accept() again for the next client.

Because requests are served strictly one after another, a slow or large transfer for one client delays every client waiting behind it; this limitation is intentional, it is exactly what Part I's performance comparison is designed to expose. The listen() backlog is set generously (e.g. SOMAXCONN) so that, even though the single-threaded server can only service one client at a time, the several concurrent clients used in the Part I experiments are queued by the kernel at the TCP handshake level rather than having their connection attempts refused while an earlier client's transfer is in progress.

5.5 Multi-Threaded Server

The multi-threaded server reuses the identical topic-lookup and file-transfer routine from Sections 5.1 to 5.3 and differs only in how a newly accepted connection is scheduled.

- The main thread owns the listening socket and does nothing but accept().
- For every accepted connection, the main thread spawns one POSIX thread (pthread_create) running a common handle_client(client_fd) routine.
- Each worker thread independently receives the request, validates the topic, transfers the files, and closes its own socket only.
- The main thread immediately loops back to accept() the next connection without waiting for any worker to finish.

```
            Server (accept loop)
             /       |       \
        Worker-1  Worker-2  Worker-3
            |         |         |
        Client-1  Client-2  Client-3
```

One thread per client (rather than a fixed thread pool) was chosen for Part I because the experiment setup only involves a handful of simultaneous clients from two client machines, and this model is easy to reason about and is reused unchanged in every Part II case, all of which are required to run on the multi-threaded server, with one thread serving each client. This assumption, that the client count stays in the tens rather than the thousands, is specific to the lab's scale; at production scale a bounded thread pool would be needed to cap thread-creation and context-switch overhead.

5.6 Common Design for Both Server Modes

The single-threaded and multi-threaded servers are not two separate implementations: they share the same topic-lookup and file-transfer code and differ only in how an accepted connection gets scheduled, handled directly by the main process, or handed off to a worker thread. Keeping this logic common matters for the Part I comparison, since it means the measured difference in throughput actually comes from concurrency and not from two subtly different transfer implementations.

5.7 Command-Line Interface

Both server modes and the client use exactly the invocation required by the assignment, so they can be driven by the automated grading harness without modification:

```
Part I server: ./server <port> <topics_root_dir>
Client:        ./client <server_ip> <server_port> <topic> <output_dir>

Example:
  ./server 5000 ./raw-img
  ./client 192.168.1.10 5000 cane ./downloads
  -> files saved under ./downloads/cane/
```

5.8 Performance Measurement Plan (Part I)

The single-threaded and multi-threaded servers are compared by varying the number of concurrently connecting clients (1, 2, 4, 8, ...), launched from the two client machines specified in the assignment's three-computer setup. For each run we record the number of concurrent clients, the total file-content bytes transmitted across all of them, and the wall-clock transfer time from the first client's connection to the last client's DONE, measured with clock_gettime(CLOCK_MONOTONIC, ...); dividing the two gives the aggregate transfer rate. Results are tabulated and plotted as aggregate transfer rate versus number of clients for both server modes on the same axes, to visualise the concurrency benefit of the multi-threaded design.

6. Part II: Fault-Tolerant File Transfer

6.1 Overview of the Three Cases

Part II extends the multi-threaded server of Part I with deliberate connection failures during a transfer, and studies three recovery strategies that trade off implementation complexity against efficiency. All three cases run on the multi-threaded server, with one thread serving each client, as required.

Case	Session state kept?	Recovery after failure
Case 1: No session management	No	Client reconnects and re-requests the whole topic from the first file.
Case 2: Session management	Yes (client + server)	Client reconnects and resumes from its last saved checkpoint (file index + byte offset).
Case 2 Enhanced	Yes, per file, per connection	Same checkpoint/resume mechanism as Case 2, applied independently across several parallel TCP connections.

6.2 Fault Injection Mechanism

To exercise the recovery logic, the server deliberately terminates some client connections while a transfer is in progress. The failure rate is controlled by a single command-line argument common to every Part II case:

```
./server <port> <topics_root_dir> <failure_probability>
Example: ./server 5000 ./raw-img 0.2   -> 20% failure probability
```

When the decision is made

Fault injection is done on the server side, because the server already controls every send() of file data and therefore knows exactly when it is safe to close a socket. The client never generates artificial failures; its only responsibility is to detect a broken connection and carry out the appropriate recovery procedure.

Rather than rolling the dice after every send() call, which would make the failure rate depend on the internal transfer-buffer size, the server checks the failure probability after a fixed amount of file data has been sent for the current connection, by default every 64 KB. This interval is a compile-time / CLI-configurable constant kept identical across Case 1 and Case 2 so the two recovery strategies are compared under the same failure conditions.

```
bytes_since_check += n;
if (bytes_since_check >= FAILURE_CHECK_INTERVAL) {
    bytes_since_check = 0;
    double r = (double) rand_r(&seed) / RAND_MAX;   // in [0,1)
    if (r < failure_probability) {
        log_fault(client_id, topic, filename, bytes_sent);
        close(client_fd);       // intentional disconnection
        return FAULT_INJECTED;
    }
}
```

Each worker thread seeds its own rand_r() state once, at thread start, as seed = (unsigned)(time(NULL) ^ (uintptr_t)pthread_self() ^ getpid()), so that two worker threads created in the same second do not draw from correlated sequences; thread-local seeding also avoids any shared-state contention or locking on the RNG itself.

This rule lets a failure occur at any point in the transfer, inside the first file, somewhere in the middle of the topic, or near the very end, which is important for testing that recovery is correct regardless of where the connection drops.

Field logged on every injected failure	Purpose
Client / connection id	Identifies which client experienced the failure.
Requested topic	Identifies the transfer being performed.
Current file & index	Shows which file was being transferred when the failure occurred.
Bytes transferred (for that file)	Shows approximately where in the file the failure occurred.

Field logged on every injected failure	Purpose
Event type	Records that the disconnection was intentionally injected, not a real network fault.

The failure-checking interval, the RNG seeding strategy above, and the default failure-probability test set (0%, 5%, 10%, 20%) are reported alongside the implementation in the final report, as required.

6.3 Case 1: No Session Management

In Case 1 neither side retains any information about an interrupted transfer. If the connection fails while a topic is being transferred, the client simply reconnects and sends the same GET <topic> request again; the server restarts the topic from its very first file. Case 1 reuses the Part I protocol unchanged (Section 5.2); no new message types are introduced, which keeps the case a clean, minimal baseline.

```
Client                                  Server
GET cane      ------------------------->
              <------------------------- OK 5
              <------------------------- FILE image1.jpeg ...
              <------------------------- FILE image2.jpeg ...
              <--- connection closed (fault injected) ---
              [reconnect]
GET cane      ------------------------->
              <------------------------- OK 5
              <------------------------- FILE image1.jpeg ... (again)
              <------------------------- ...
              <------------------------- DONE
```

Client behaviour after failure

- Detect the failure: a recv() returns 0 or an error before DONE has been received.
- Close the failed socket and discard every file byte received during that attempt.
- Reconnect to the server and send GET <topic> again.
- Receive the complete topic from the first file once more.

Files are written to a temporary, per-attempt location and only moved into output_dir/topic/ once the whole topic transfer completes successfully; a failed attempt's partial files are discarded rather than mixed with a later successful attempt.

Server behaviour

The server keeps no per-client recovery state at all: current file, byte offset, and completed-file information are neither stored nor consulted. Every new connection, even from a client that just failed, is treated as an entirely new request and always starts sending from the first file of the topic.

Retry behaviour

Connection failures caused by fault injection are expected in Part II, so the client reconnects automatically and resends the same GET <topic> request; it keeps doing this until the topic is fully received (or a configured retry limit is hit, to avoid looping forever if the server has gone down). This

automatic retry is only appropriate for a connection failure, though; a genuine application-level error such as ERROR UNKNOWN_TOPIC means the request itself was invalid, and retrying it would just get the same error again. The client checks which of the two happened before deciding whether to reconnect or simply report the problem and stop.

Expected overhead

Every experiment tracks total bytes transmitted, useful bytes (the size of the final, successfully delivered topic), redundant bytes (transmitted minus useful), completion time, and throughput, so that Case 1 can be compared directly against Case 2 at the same failure probability. Because every failure here discards all progress, Case 1's redundant-byte count is expected to grow sharply as the failure probability increases; this is the baseline against which Case 2's checkpointing is evaluated in Section 7.

6.4 Case 2: Session Management and Resume

Case 2 keeps enough state on both sides that, after a reconnection, the transfer can resume from the last confirmed byte instead of restarting the topic.

6.4.1 Checkpoint information

A checkpoint records exactly the fields the assignment suggests, the topic, the current file, and a byte offset, plus the set of files already completed:

Field	Purpose
Topic	Identifies the requested topic.
File index	Position of the current file within the topic's file list (0-based, in a fixed, sorted order agreed by client and server).
Current file name	Identifies which file is in progress.
Byte offset	Number of bytes of the current file already confirmed as received.
Completed files	Indicates which earlier files in the topic are already fully received.

6.4.2 Where the state lives

The client is treated as the authoritative source of "bytes successfully received", because it is the one that actually writes bytes to the output file; bytes the server merely attempted to send() may not have arrived before a sudden disconnect. The server independently keeps a lightweight session table so it can identify and validate a resume request without trusting the client blindly on the parts that can be checked statically (see the four checks in Section 6.4.6): whether the session exists, is still active, and whether the requested file index and offset are within the bounds the server itself already knows.

Side	State kept
Client	Session id; per-file checkpoint (file index, byte offset, completed-file bitmap); the partially written output file itself, kept rather than deleted.
Server	Session table entry: session id -> {topic, current file index, completed-file

Side	State kept
	bitmap}, protected by a mutex because the server is multi-threaded.

Design note on the offset field: the server's session-table entry intentionally does not store its own independently-tracked "last acknowledged byte offset" to cross-check against the offset the client reports in RESUME. This is a deliberate choice, not an oversight: on every RESUME the server always resumes sending from the byte offset the client itself supplies, since that value is, by design, the client's authoritative record of what it has actually written to disk (Section 6.4.2, first paragraph). A server-side shadow counter of "bytes sent" would not be more trustworthy than the client's figure, because bytes the server sent may never have arrived; it would only add bookkeeping without improving correctness. The two checks the server does perform, that the offset does not exceed the on-disk file size and that the file is not already marked completed (Section 6.4.6), are exactly the ones needed to keep a resume request safe against a stale or corrupted client-side checkpoint.

6.4.3 Session identification

When a client starts a new topic transfer, the server allocates a session id (a simple, non-cryptographic counter; it only needs to be unique among currently active sessions, not secure) and returns it to the client. The client presents this id on every subsequent CHECKPOINT and RESUME message so the server can find the right session-table entry under its mutex.

6.4.4 Message format: Case 2 (fixed-length messages)

Case 2 reuses the fixed-length message_header struct from Section 5.2 and extends it with session-related fields (also fixed-width, network-byte-order integers), rather than switching to text lines:

```
struct message_header {
    char     method[METHOD_LEN];
    char     topic[TOPIC_LEN];
    char     filename[FNAME_LEN];
    uint32_t num_files;
    uint64_t filesize;
    uint64_t offset;           // start_offset (FILE) / confirmed offset
(CHECKPOINT, RESUME)
    uint32_t session_id;       // used from SESSION onward
    uint32_t file_index;       // used by CHECKPOINT / RESUME
    uint16_t error_code;       // used by ERROR (e.g. INVALID_SESSION,
INVALID_OFFSET)
};
```

Message (method)	Direction	Purpose
GET	Client -> Server	Starts a brand-new topic transfer.
SESSION	Server -> Client	session_id assigns an id to the new transfer.
OK	Server -> Client	num_files confirms the topic and file count.
FILE	Server -> Client	filename, filesize and offset announce a file, its size, and the byte offset the data below starts at (0 for a fresh file).
<raw file bytes>	Server -> Client	File content beginning at offset.

Message (method)	Direction	Purpose
CHECKPOINT	Client -> Server	session_id, file_index, offset periodically report confirmed progress (e.g. every 64 KB, and whenever a file completes).
RESUME	Client -> Server	session_id, file_index, offset, sent instead of GET after reconnecting; asks the server to continue from this point.
RESUME_OK	Server -> Client	The checkpoint was validated; the server will now resume sending.
ERROR	Server -> Client	error_code = INVALID_SESSION or INVALID_OFFSET; the resume request failed validation (Section 6.4.6), client falls back to a fresh GET.
DONE	Server -> Client	The complete topic has been transferred; both sides discard the session.

6.4.5 Normal transfer and mid-transfer failure

```
Client                                  Server
GET cane          ------------------------>
                  <------------------------ SESSION 12
                  <------------------------ OK 5
                  <------------------------ FILE image1.jpeg 45231 0
                  <------------------------ <45231 bytes>
CHECKPOINT 12 0 45231  ------------------->
                  <------------------------ FILE image2.jpeg 800000 0
                  <------------------------ <262144 bytes>
CHECKPOINT 12 1 262144 ------------------->
                  <--- connection closed (fault injected) ---
                  [reconnect]
RESUME 12 1 262144     ------------------->
                  <------------------------ RESUME_OK
                  <------------------------ FILE image2.jpeg 800000 262144
                  <------------------------ <537856 remaining bytes>
CHECKPOINT 12 1 800000 ------------------->
                  <------------------------ ... (remaining files) ...
                  <------------------------ DONE
```

6.4.6 Checkpoint validation

The server never trusts a resume request blindly. Before honouring RESUME, it checks, in order: (1) the session id exists and belongs to an active, not-already-completed transfer; (2) the requested file index is within range for that topic; (3) the requested byte offset does not exceed the size of that file; and (4) the file at that index has not already been marked completed in the session table. As noted in Section 6.4.2, the server does not additionally cross-check the offset against a server-side sent-byte counter: the client's reported offset is treated as authoritative once it passes these four structural checks, since it reflects what was actually written to disk. If any check fails, the server replies with an ERROR message instead of seeking into the file, and the client falls back to sending a fresh GET <topic>, which is always safe.

6.4.7 Checkpoint frequency: a trade-off

A checkpoint is recorded after every fixed amount of successfully received data (64 KB by default) and whenever a file completes. A smaller interval reduces the data that must be retransmitted after a failure but adds more CHECKPOINT round-trips; a larger interval does the opposite. 64 KB was chosen as a starting point that keeps checkpoint traffic well under 1% of a typical multi-hundred-KB image file, and remains a tunable constant.

Case 1 vs. Case 2, at a glance

Feature	Case 1	Case 2
Progress stored	No	Yes
Partial file kept after failure	No	Yes
Completed files reused	No	Yes
Resume from a byte offset	No	Yes
Restart point after failure	Beginning of topic	Last checkpoint
Protocol complexity	Lower	Higher

What Case 2 improves

Case 2 is measured on the same axes as Case 1, total, useful and redundant bytes, completion time and throughput, plus one extra figure specific to this case: checkpoint overhead, the extra bytes and round-trips spent on CHECKPOINT and RESUME messages. Because a resumed transfer only re-sends data since the last confirmed checkpoint rather than the whole topic, Case 2's redundant-byte count is expected to stay far smaller than Case 1's at every failure probability tested.

6.5 Case 2 Enhanced (Parallel TCP Connections)

The Enhanced case keeps Case 2's checkpoint/resume mechanism unchanged in spirit and adds the required performance enhancement: transferring the files of one topic over several simultaneous TCP connections instead of one.

6.5.1 Parallelism granularity

Parallelism is applied at the file level, not by splitting a single file into byte ranges across connections. Each worker connection owns exactly one file at a time from a shared, mutex-protected work queue; when it finishes, it pulls the next available file index from the queue. File-level parallelism was preferred because the dataset already contains many independent files, it needs no byte-range reassembly logic, and it makes failure recovery simple: only the one file a failed connection was working on needs to be resumed.

6.5.2 Connection roles

A control connection is opened first to negotiate the session and obtain the file list; N worker connections are then opened to actually transfer file data in parallel (N = 4 by default). N is a client-

side, optional trailing command-line argument to the client program (Section 6.5.8 and Section 11); the server does not need to know N in advance, since every worker connection independently identifies itself with the shared session_id once opened.

```
                    Server
    control  /        |        |        \  workers
            /         |        |         \
  Client -- TCP1 --- TCP2 --- TCP3 --- TCP4
         (control)    W1       W2       W3   (W4 similar)
```

6.5.3 Session state: extended for multiple in-flight files

Because several files of the same topic are now in flight concurrently under one session id, one per worker connection, the Case 2 session-table entry of Section 6.4.2 (a single {topic, current file index, completed-file bitmap} record) is not sufficient on its own: it must track one in-progress-file record per active worker, not just one file. The Enhanced-case server therefore keeps, per session, a small map keyed by file index rather than a single current-file field:

```
struct enhanced_session {
    uint32_t session_id;
    char     topic[TOPIC_LEN];
    uint32_t num_files;
    uint8_t  completed[MAX_FILES];              // 1 = file fully delivered
    struct {                                      // one entry per file
        uint8_t  in_progress;                     // 1 while a worker owns it
        uint64_t last_checkpoint_offset;
    } file_state[MAX_FILES];
    pthread_mutex_t lock;                         // guards this whole entry
};
```

This is an explicit extension of the Case 2 schema (Section 6.4.2), not a reuse of it unchanged: the single current_file_index/offset pair becomes a per-file_index entry, indexed by file_index, so that concurrent CHECKPOINT and FILE_REQUEST messages from different workers on the same session never overwrite each other's progress record. All reads and writes to a session's file_state array happen under that session's own mutex.

6.5.4 Message format: Case 2 Enhanced (fixed-length messages)

As in Case 2, every message is the same fixed-length message_header struct (Section 6.4.4); FILELIST_ENTRY messages are simply sent as num_files repetitions of one fixed-size header each, rather than a variable-length list, so the receiver always knows how many fixed-size records to read once it has seen num_files in SESSION.

Message (method)	Conn.	Direction	Purpose
GET	Control	Client -> Server	Requests the topic and its file listing.
SESSION	Control	Server -> Client	session_id, num_files assign the overall session id and file count.
FILELIST_ENTRY	Control	Server -> Client	file_index, filename, filesize; one message per file, repeated num_files times.

Message (method)	Conn.	Direction	Purpose
FILELIST_END	Control	Server -> Client	Marks the end of the listing; control connection may then close.
FILE_REQUEST	Worker	Client -> Server	session_id, file_index, offset request one specific file, from a given byte offset (0 fresh, >0 resume).
FILE	Worker	Server -> Client	filename, filesize, offset confirm the file and starting offset before streaming its bytes.
<raw file bytes>	Worker	Server -> Client	The file content starting at offset.
CHECKPOINT	Worker	Client -> Server	session_id, file_index, offset: periodic progress report for the file that worker owns.
FILE_DONE	Worker	Client -> Server	file_index confirms that file is fully received; worker is now free for the next queued file.
SESSION_DONE	Control	Client -> Server	Sent once the client has received FILE_DONE-worthy confirmation for every file in the topic; tells the server to discard the session-table entry.
ERROR	Any	Server -> Client	error_code identifies an invalid request / session / offset on that connection.

SESSION_DONE closes the gap left by the per-file FILE_DONE messages: FILE_DONE only tells the server that one file is complete and that its owning worker connection is free for reuse, but nothing in Case 2's own DONE message maps cleanly onto "the whole multi-connection topic transfer is finished." The client already knows locally when every file index it received in FILELIST_ENTRY has been confirmed complete (it is simply counting FILE_DONE confirmations against num_files), so it is the natural side to signal completion; it reopens (or reuses) the control connection to send SESSION_DONE, and the server evicts the session-table entry on receipt, exactly mirroring how DONE causes both sides to discard the session in Case 2 (Section 6.4.4). A session that never receives SESSION_DONE, for example because the client crashed after its last file completed, is still reclaimed by the idle-session reaper described in Section 10.3.

6.5.5 Work distribution

Rather than statically pre-assigning a fixed list of files to each connection, the client maintains one shared queue of pending file indices. Whenever a worker becomes idle it pops the next index and sends FILE_REQUEST for it. This queue-based distribution is used because the animal images vary in size; static, count-based partitioning could leave one worker with several large files while another sits idle after finishing small ones early.

6.5.6 Independent failure recovery per connection

Fault injection continues to apply independently to each active server-side connection (Section 6.2). If one worker connection is dropped, only the file that worker was transferring is affected; the remaining workers keep transferring their own files uninterrupted. The affected worker simply reconnects and

sends FILE_REQUEST with its last CHECKPOINT for that file, exactly as Case 2's RESUME does, just addressed to one file rather than the whole topic, and reads the resume offset from that file's entry in the session's file_state map (Section 6.5.3):

```
Connection 1 -> File 1 -> completed
Connection 2 -> File 2 -> connection failure
Connection 3 -> File 3 -> transferring (unaffected)
Connection 4 -> File 4 -> transferring (unaffected)

Connection 2 reconnects:
  FILE_REQUEST 21 2 262144  ------------>
                <------------------------ FILE image2.jpeg 800000 262144
                <------------------------ <537856 bytes>
  FILE_DONE 2   ------------------------->
```

6.5.7 Reassembly

Because parallelism is file-level, no byte-level reassembly is required: every worker writes its own file directly to output_dir/topic/<filename>, and the files never share a destination. The topic transfer is considered complete only once every file index has been reported as FILE_DONE (Section 6.5.4), at which point the client sends SESSION_DONE.

6.5.8 Blocking vs. non-blocking sockets

The implementation uses blocking sockets, one thread per worker connection, on both the client and the server. Each thread can block on its own send()/recv() without stalling the other connections, since they already run concurrently in separate threads. Non-blocking sockets with select()/poll() were considered but rejected for the first iteration: they would require additional state machines to track each connection's progress and would complicate the recovery logic without a clear benefit, given that thread-per-connection already provides the desired concurrency.

6.5.9 Number of parallel connections

Four connections is used as the default, set on the client side as an optional trailing command-line argument (Section 11): ./client <server_ip> <server_port> <topic> <output_dir> [num_connections]. Too few connections under-uses the available concurrency; too many increases thread-creation and context-switch overhead and may saturate the network with little further gain. One connection behaves like plain Case 2 and is a useful baseline for isolating the effect of parallelism; 2 to 4 connections are expected to noticeably reduce completion time by overlapping several file transfers; 8 or more risk diminishing or negative returns once thread and socket overhead dominates. The exact trade-off is measured experimentally by comparing 1, 2, 4 and 8 connections (Section 7).

Expected outcome: because parallelism here is bounded by the number of files in a topic rather than by their total byte count, completion time is expected to fall roughly in proportion to min(N, file_count) as N grows from 1 upward, with the benefit flattening once N approaches the topic's file count (10 for most Animals-10 topics) and turning negative once thread/socket overhead exceeds the per-file transfer time saved. This is the specific hypothesis Section 7's connection-count sweep (1, 2, 4, 8) is designed to confirm or refute.

7. Consolidated Performance Measurement Plan

All experiments run on the three-computer setup required by the assignment: one machine runs the server, and client processes run on the other two machines. Every experiment uses the same dataset and comparable test conditions.

Experiment	Independent variable	Key metrics
Part I	Number of concurrent clients (1, 2, 4, 8, ...)	Aggregate transfer rate vs. number of clients, single- vs. multi-threaded.
Case 1 vs. Case 2	Failure probability (0, 5%, 10%, 20%, ...)	Total / useful / redundant bytes, completion time, throughput.
Case 2 vs. Case 2 Enhanced	Number of parallel connections (1, 2, 4, 8)	Completion time, throughput, checkpoint/recovery overhead.

Timing starts when the client begins the topic transfer and stops once the complete topic has been verified on disk, measured with a monotonic clock (clock_gettime(CLOCK_MONOTONIC, ...)) so wall-clock adjustments cannot skew results. All raw measurements are written to CSV files under a Results/ directory and used to generate the required tables and graphs (e.g. throughput vs. number of clients, redundant bytes vs. failure probability), using the common schema below.

```
case, failure_probability, failures, total_bytes, useful_bytes,
redundant_bytes, completion_time, throughput
```

8. Program Structure and Additional Features

8.1 Folder / Module Structure

The final submission uses one self-contained folder per case, each independently buildable with its own Makefile, exactly as the assignment requires:

```
FinalSubmission_Group_040_093/
|-- FinalReport_Group_040_093.pdf
|-- README.md
|-- Part1/
|     |-- server.c client.c common.c/h transfer.c/h Makefile
|-- Part2_Case1/
|     |-- server.c client.c common.c/h transfer.c/h fault.c/h Makefile
|-- Part2_Case2/
|     |-- server.c client.c common.c/h transfer.c/h fault.c/h
|     |-- session.c/h Makefile
|-- Part2_Case2_Enhanced/
|     |-- server.c client.c common.c/h transfer.c/h fault.c/h
|     |-- session.c/h worker.c/h Makefile
`-- Results/
      |-- *.csv *.png
```

Each case folder builds exactly two executables, server and client, with make, and every case reuses the previous case's networking and transfer code rather than being written from scratch:

Part I -> Case 1 -> Case 2 -> Case 2 Enhanced

- Part I contributes the listening socket, topic table, the fixed-length message_header struct, and the send_all/recv_all framed transfer routine.
- Case 1 adds fault.c (fault injection) and the client's reconnect-and-restart loop; the transfer routine itself is untouched.
- Case 2 adds session.c (session table, checkpoint struct, validation) and the CHECKPOINT / RESUME message handling on both sides.
- Case 2 Enhanced adds worker.c (shared file queue, one thread per worker connection) and extends session.c's schema to the per-file file_state map of Section 6.5.3, on top of Case 2's checkpoint logic.

8.2 Key Functions

Server

Function	Role
create_server_socket()	Create, bind and listen on the TCP socket (backlog set to SOMAXCONN).
load_topics(root_dir)	Scan the root directory and populate topic_table.
find_topic(name)	Linear lookup of a topic by name.
handle_client(fd)	Top-level per-connection routine, run directly (Part I single-threaded) or inside a worker thread (everywhere else).
send_file(fd, path, offset)	Send the FILE header and the file bytes starting at offset.
maybe_inject_fault(fd, ctx)	Byte-interval failure check described in Section 6.2.
create_session() / find_session() / update_checkpoint()	Session-table management (Case 2, Case 2 Enhanced).
validate_resume(session, file_idx, offset)	The four checks of Section 6.4.6.
log_event(...)	Structured logging for connections, faults, resumes and completions.

Client

Function	Role
connect_to_server(ip, port)	Create and connect a TCP socket; reused for both the initial attempt and every reconnect.
send_get(topic) / send_resume(session, idx, offset)	Build and send the opening fixed-length request for the current case.

Function	Role
receive_file(fd, out_path, expected_size, offset)	Receive exactly the expected bytes and append/write them at offset.
save_checkpoint() / load_checkpoint()	Persist the client-side progress record described in Section 6.4.1.
run_transfer(...)	Drives one topic transfer end-to-end, including the reconnect loop on failure.
worker_thread(...)	Enhanced-case only: repeatedly pulls a file index from the shared queue and transfers it over its own connection.
record_stats(...)	Accumulates bytes transferred and elapsed time for the performance report.

8.3 Additional Features Beyond the Minimum

Logging

Both programs log connection/disconnection events, the requested topic, injected faults (with the fields listed in Section 6.2), reconnection and resume attempts, checkpoint creation, and total bytes/time for each transfer. This is essential for verifying, independently of the final numbers, that a resumed transfer really did continue from the expected byte offset, and it feeds directly into the performance write-up.

Configurability

Port, dataset root, server IP and failure probability are always supplied on the required command-line arguments. Implementation-level tunables that are not part of the required interface, the failure-checking interval, the checkpoint interval, and the number of parallel connections in the Enhanced case, are given sensible compiled-in defaults and may optionally be overridden with extra trailing command-line arguments, so the two mandatory invocation forms always continue to work unmodified.

Graceful shutdown

The server installs a handler for SIGINT/SIGTERM that stops accept()-ing new connections, closes the listening socket, and lets in-flight worker threads finish (or joins them with a bounded timeout) before releasing shared structures such as the session table and its mutex. This avoids leaking sockets, file descriptors or memory during long, repeated fault-injection experiments.

Robust error handling

Return values of every socket, file and threading call are checked. Distinct, clearly logged error paths exist for: invalid command-line arguments, socket/bind/listen failures, an unknown topic, a file that cannot be opened, an interrupted connection, an invalid resume offset or session id, an over-length field rejected during fixed-length message parsing, and a failure to create the client's output directory. Errors never leave the program silently continuing with an inconsistent state.

Extensibility to new topics

Because the server discovers topics by scanning topics_root_dir at start-up rather than hard-coding a list, dropping a new sub-directory (e.g. lion/) into the dataset root makes it available as a new topic the next time the server starts, with no source-code change.

Resource cleanup

Every socket and file descriptor is closed as soon as it is no longer needed. Each worker thread is responsible for the resources of its own connection only, and session/checkpoint memory is freed once a topic transfer completes (or is abandoned) so that long-running, repeated-failure experiments do not leak memory or leave sockets in CLOSE_WAIT.

9. Design Thought Process and Alternatives Considered

This section summarises the main alternatives weighed for each part of the design and the reasoning behind the choice actually made.

Decision	Alternative(s) considered	Chosen approach	Reason
Topic -> directory mapping	Hash table	Simple array, linear search	Only ~10 topics; lookup cost is negligible next to transfer time.
Part I concurrency model	Fixed thread pool	One thread per client	Matches the assignment's suggested design and is reused unchanged in Part II, at the lab's expected client-count scale.
Wire protocol style	Newline-terminated ASCII control lines	Fixed-length binary message_header struct (method code + zero-padded fields)	Every message has a known size, so the receiver never scans for a delimiter; field widths are fixed at compile time, which also makes bounds-checking straightforward and removes an entire class of parsing ambiguity.
Fault-injection trigger	Random check after every send()	Random check after a fixed byte interval	Keeps the failure rate independent of the internal buffer size, so cases are compared fairly.

Decision	Alternative(s) considered	Chosen approach	Reason
Case 1 recovery	Keep completed files, resume from next file	Discard everything, restart from file 1	Keeping progress would already be session management, defeating the purpose of a clean baseline.
Case 2 checkpoint granularity	Checkpoint only per whole file	Checkpoint at fixed byte intervals and per file	Allows resuming inside a large, partially received file instead of re-sending it whole.
Case 2 checkpoint authority	Server-side byte count, cross-checked against the client's reported offset	Client-confirmed offset, trusted once it passes structural validation	The server only knows what it attempted to send; the client knows what was actually written to disk, so a server-side shadow count would add bookkeeping without improving correctness (Section 6.4.2).
Enhanced session-table schema	Reuse Case 2's single-current-file record unchanged	Per-file_index map of in-progress state within one session entry	Several files are in flight under one session id at once in the Enhanced case; a single current-file field cannot represent that, so the schema is explicitly extended (Section 6.5.3).
Enhanced parallelism granularity	Byte-range split of one file	Whole files from a shared work queue	Avoids merging byte ranges, keeps synchronisation simple, isolates one connection's failure to one file.
Enhanced socket model	Non-blocking + select()/poll()	Blocking sockets, one thread per connection	Thread-per-connection already gives the needed concurrency without an extra event-driven state machine.
Parallel connection count	Fixed at 1, or unbounded	Configurable client-side argument, default 4, measured against 1/2/4/8	Balances speed-up against overhead; the best value is determined

Decision	Alternative(s) considered	Chosen approach	Reason
			experimentally, not assumed.

Overall Progression

The implementation is built incrementally rather than as four unrelated programs, so that later cases reuse, and do not risk breaking, the tested logic of earlier ones:

basic file transfer -> concurrent transfer -> failure injection and restart -> checkpoint-based resume -> parallel-connection enhancement

This progression also gives a clean basis for the required performance comparisons, because each stage adds exactly one new capability on top of an already-working and already-measured previous stage.

10. Assumptions, Limitations and Edge Cases

The following assumptions and boundary behaviours are fixed here so that every group member implements the same contract, and so the auto-grader sees consistent behaviour across edge cases the assignment brief does not spell out explicitly.

10.1 Assumptions

- The dataset root directory is readable at server start-up and does not change shape (topics added or removed) while the server is running; a topic is identified purely by its directory name, which is assumed to fit within TOPIC_LEN and to be a valid, printable string with no embedded whitespace or path separators.
- File names within a topic are unique, fit within FNAME_LEN, and contain no path-traversal sequences (e.g. "../"); the server treats the topic directory as a flat listing of regular files and does not recurse into further sub-directories.
- The client's output_dir is writable and has enough free space for the requested topic; the client creates output_dir/<topic>/ if it does not already exist.
- Session ids (Case 2 / Enhanced) are unique only among concurrently active sessions on one server process; they are not persisted across a server restart, so a server crash invalidates all in-flight sessions and every affected client must fall back to a fresh GET.
- The three-computer test setup places the server and both client machines on the same local network, so propagation delay is small relative to file-transfer time; the fault-injection interval (64 KB) and checkpoint interval were chosen with this in mind.
- The one-thread-per-client model (Section 5.5) is adequate at the lab's expected scale of a handful of simultaneous clients from two client machines; it is not assumed to hold at much larger client counts.

10.2 Known Limitations

- A single, non-cryptographic counter is used for session ids; this is adequate for a controlled lab experiment but would need a harder-to-guess identifier (and authentication) before use on an untrusted network.
- The session table currently lives only in server memory, so a server restart mid-experiment loses all resumable state; an acceptable trade-off given the assignment's scope, but worth noting as a direction for future work (e.g. a lightweight on-disk session log).
- Fault injection only closes the socket; it does not model partial writes, reordering, or corruption within a still-open connection, so it exercises the reconnect/resume path but not TCP-level retransmission behaviour (which Lab 5 already studies separately in NetSim).
- File-level parallelism in the Enhanced case means the speed-up from additional connections is bounded by the number and size distribution of files in a topic; a topic with very few, very large files would see less benefit from N greater than the file count (Section 6.5.9).
- One thread per accepted connection has no upper bound in the current design; this is acceptable at the lab's tested client counts (up to 8) but would need a bounded thread pool to remain safe if client counts grew by orders of magnitude.

10.3 Edge Cases Handled by Design

Edge case	Handling
Empty topic directory (0 files)	Server replies OK with num_files = 0 and immediately DONE; client creates an empty output_dir/<topic>/ and reports success.
Unknown topic name	Server replies ERROR UNKNOWN_TOPIC without opening any file or allocating a session.
Resume request for an already-completed file	Rejected by validate_resume() (Section 6.4.6, check 4); client falls back to a fresh GET.
Resume offset beyond end-of-file	Rejected by validate_resume() (Section 6.4.6, check 3), which guards against a corrupted or stale client-side checkpoint.
Client crashes mid-transfer, never reconnects	Server's session-table entry becomes stale; a periodic reaper thread (or a lazy check on the next session-table lookup) evicts sessions idle beyond a timeout so memory is not leaked indefinitely; this also reclaims Enhanced-case sessions that never received SESSION_DONE (Section 6.5.4).
Two workers in the Enhanced case finish at the same instant	The shared work queue is protected by a mutex, so queue pops are serialised and no file index is ever handed to two workers at once.
Over-length topic name, filename, or malformed method code in an incoming message	Rejected during fixed-length field parsing (Section 5.1) before any lookup or file operation is attempted.

11. Build, Execution and CLI Compliance

Each case folder contains its own Makefile. Running make inside it produces exactly two executables, server and client, in that same folder, with no interactive prompts required after start-up.

Program	Required invocation
Part I server	./server <port> <topics_root_dir>
Part II server (all cases)	./server <port> <topics_root_dir> <failure_probability>
Client (Part I, Case 1, Case 2)	./client <server_ip> <server_port> <topic> <output_dir>
Client (Case 2 Enhanced)	./client <server_ip> <server_port> <topic> <output_dir> [num_connections]

No file path, port number or IP address is ever hard-coded. The num_connections argument on the Enhanced client is the only case-specific extra argument in this design (Section 6.5.9); it is optional and defaults to 4, so the mandatory four-argument invocation above continues to work unchanged for automated testing across every case.

12. Conclusion

This design provides a consistent progression from a basic TCP topic-distribution service to a fault-tolerant, concurrent, multi-connection system. Part I establishes the baseline protocol, using fixed-length messages throughout, and studies sequential versus concurrent servicing. Part II introduces controlled failures and evaluates restart-based recovery (Case 1) against checkpoint-based resume (Case 2), then extends Case 2 with parallel TCP connections and a per-file session schema (Case 2 Enhanced). Every message format, data structure and design trade-off documented here is intended to be implemented directly, with the final report recording the measured results and comparing them against the expectations set out in this document.